

SETTINGS AND PERMISSIONS

Configure the sandboxed Bash tool

Learn how Claude Code's sandboxed Bash tool provides filesystem and network isolation for safer, more autonomous agent execution.

The Bash sandbox lets Claude run most shell commands without stopping to ask permission. Instead of approving each command, you define which files and network domains commands can touch, and the operating system enforces that boundary for every Bash command and its child processes.

! To compare other isolation approaches such as dev containers, custom containers, and virtual machines, see [Sandbox environments](#). To reduce permission prompts for tools other than Bash, see [permission modes](#).

Get started

The sandbox is built into Claude Code and runs on macOS, Linux, and WSL2. Native Windows is not supported. On Windows, run Claude Code inside a WSL2 distribution.

On macOS, there is nothing to install: sandboxing uses the built-in Seatbelt framework. On Linux and WSL2, the sandbox relies on two packages, covered in [Set up Linux and WSL2](#). Even if you haven't installed them yet, you can start with `/sandbox`, because its panel shows whether anything is missing.

1 Run `/sandbox`

Start a Claude Code session and run the `/sandbox` command:

```
/sandbox
```

This opens the sandbox panel with three tabs:

- **Mode:** choose how sandboxed commands are approved, covered in the next step

- **Overrides:** choose whether commands that fail under the sandbox can fall back to running unsandboxed. This is the `allowUnsandboxedCommands` setting
- **Config:** view the resolved sandbox settings

If the panel shows only a Dependencies tab, a required package is missing. Install it as described in [Set up Linux and WSL2](#), restart Claude Code, and run `/sandbox` again.

2 Choose a mode

On the Mode tab, select auto-allow or regular permissions. Auto-allow runs sandboxed commands without prompting, and regular permissions keeps the regular permission prompts even when commands are sandboxed. See [Sandbox modes](#) for which commands still prompt in auto-allow mode.

3 Run a Bash command

Ask Claude to run a command, such as a build or a test suite. By default, commands inside the sandbox can write only to the working directory and the session temp directory. The first time a command needs a new network domain, Claude Code prompts for approval.

Commands that cannot run sandboxed fall back to the regular permission flow. To widen or narrow these boundaries, see [Configure sandboxing](#).

Selecting a mode in the panel writes to your project's local settings at

`.claude/settings.local.json`, which apply to the current project and are not checked into git. To enable the sandbox across all of your projects, set `sandbox.enabled` to `true` in your user settings at `~/.claude/settings.json`. To enforce sandboxing for every developer in an organization, use [managed settings](#).

⚠ By default, if the sandbox cannot start because dependencies are missing or the platform is unsupported, Claude Code shows a warning and runs commands without sandboxing. To make this a hard failure instead, set `sandbox.failIfUnavailable` to `true`. This is intended for managed deployments that require sandboxing as a security gate.

Set up Linux and WSL2

On Linux and WSL2, the sandbox relies on two packages:

- [bubblewrap](#): the unprivileged sandboxing tool that enforces filesystem isolation

- socat : the relay used to route network traffic through the sandbox proxy

Install them with your distribution's package manager:

Ubuntu/Debian

```
sudo apt-get install bubblewrap socat
```

Fedora

```
sudo dnf install bubblewrap socat
```

After installing, the Dependencies tab in `/sandbox` shows whether `ripgrep` , `bubblewrap` , `socat` , and the seccomp filter are available on your platform. Ripgrep is bundled with the native Claude Code binary. The seccomp filter is optional and adds Unix domain socket blocking. Install it with `npm install -g @anthropic-ai/sandbox-runtime` if it is missing.

When a required dependency is missing, the Dependencies tab is the only tab shown until you install it. The dependency check runs at startup, so restart Claude Code after installing packages for `/sandbox` to detect them.

▸ Ubuntu 24.04 and later: allow bubblewrap to create user namespaces

▸ WSL2 notes

Sandbox modes

Claude Code offers two sandbox modes:

Auto-allow mode: Bash commands will attempt to run inside the sandbox and are automatically allowed without requiring permission. Commands that cannot be sandboxed, such as those needing network access to non-allowed hosts, fall back to the regular permission flow, where Claude Code

checks your permission rules and prompts you for any command those rules do not already allow.

Even in auto-allow mode, the following still apply:

- Explicit deny rules are always respected
- `rm` or `rmdir` commands that target `/`, your home directory, or other critical system paths still trigger a permission prompt
- Content-scoped ask rules like `Bash(git push *)` still force a prompt even for sandboxed commands
- A bare `Bash` ask rule, or the equivalent `Bash(*)` form, is skipped for commands that run sandboxed; it still applies to commands that fall back to the regular permission flow

Regular permissions mode: All Bash commands go through the regular permission flow, even when sandboxed. This provides more control but requires more approvals.

In both modes, the sandbox enforces the same filesystem and network restrictions. The difference is only in whether sandboxed commands are auto-approved or require explicit permission.

The session temp directory is writable inside the sandbox by default, alongside the working directory. Claude Code sets `$TMPDIR` to this directory for sandboxed commands, so tools that write temporary files work without extra configuration. Unsandboxed commands inherit your shell's `$TMPDIR` unchanged, which means sandboxed and unsandboxed commands resolve `$TMPDIR` to different directories. To pass temporary files between the two, write them under the working directory instead.

Some commands cannot run inside the sandbox at all, such as tools that are incompatible with it or that need a host you have not allowed. Rather than failing the task or requiring you to turn sandboxing off, Claude Code includes an escape hatch: when a command fails because of sandbox restrictions, Claude analyzes the failure and may retry the command with the `dangerouslyDisableSandbox` parameter. The retried command runs outside the sandbox, so it goes through the regular permission flow and requires your approval.

You can disable this escape hatch by setting `"allowUnsandboxedCommands": false` in your sandbox settings. When disabled, which the `/sandbox` Overrides tab shows as **Strict sandbox mode**, the `dangerouslyDisableSandbox` parameter is completely ignored and all commands must run sandboxed or be explicitly listed in `excludedCommands`.

- ❗ Auto-allow mode works independently of your permission mode setting. Even if you're not in "accept edits" mode, sandboxed Bash commands will run automatically when auto-allow is enabled. This means Bash commands that modify files within the sandbox boundaries will execute without prompting, even when file edit tools would normally require approval.

Configure sandboxing

Customize sandbox behavior through your `settings.json` file. See [Settings](#) for the complete configuration reference.

By default, sandboxed commands can write only to the current working directory and the session temp directory. If subprocess commands like `kubectl`, `terraform`, or `npm` need to write outside those directories, use `sandbox.filesystem.allowWrite` to grant access to specific paths:

```
{
  "sandbox": {
    "enabled": true,
    "filesystem": {
      "allowWrite": ["~/.kube", "/tmp/build"]
    }
  }
}
```

These paths are enforced at the OS level, so all commands running inside the sandbox, including their child processes, respect them. This is the recommended approach when a tool needs write access to a specific location, rather than excluding the tool from the sandbox entirely with `excludedCommands`.

When the same filesystem array is defined in multiple [settings scopes](#), the arrays are merged: paths from every scope are combined, not replaced.

Path prefixes control how paths are resolved:

Prefix	Meaning	Example
/	Absolute path from filesystem root	/tmp/build stays /tmp/build
~/	Relative to home directory	~/.kube becomes \$HOME/.kube
./ or no prefix	Relative to the project root for project settings, or to ~/.claude for user settings	./output in .claude/settings.json resolves to <project-root>/output

This syntax differs from **Read and Edit permission rules**, which use //path for absolute and /path for project-relative. Sandbox filesystem paths use standard conventions: /tmp/build is absolute.

You can also deny write or read access using sandbox.filesystem.denyWrite and sandbox.filesystem.denyRead , and re-allow specific paths within a denied region using sandbox.filesystem.allowRead .

The example below blocks reading from the entire home directory while still allowing reads from the current project. Place it in your project's .claude/settings.json , because the relative path . resolves to the project root only when the configuration lives in project settings:

```
{
  "sandbox": {
    "enabled": true,
    "filesystem": {
      "denyRead": ["~/"],
      "allowRead": ["."]
    }
  }
}
```

The . in allowRead resolves to the project root because this configuration lives in project settings. If you placed the same configuration in ~/.claude/settings.json , . would resolve to ~/.claude instead, and project files would remain blocked by the denyRead rule.

Protect credentials

The sandbox.credentials setting declares credential files and environment variables that sandboxed commands must not access. Listed file paths are denied for reads inside the sandbox, the same restriction that filesystem.denyRead applies, and listed environment variables are unset before each sandboxed command runs. The dedicated credentials block keeps credential rules

grouped with the environment variables to unset and separate from general filesystem rules. Requires Claude Code v2.1.187 or later.

The example below blocks reads of the AWS credentials file and the SSH directory and removes `GITHUB_TOKEN` and `NPM_TOKEN` from the environment of sandboxed commands:

```
{
  "sandbox": {
    "enabled": true,
    "credentials": {
      "files": [
        { "path": "~/.aws/credentials", "mode": "deny" },
        { "path": "~/.ssh", "mode": "deny" }
      ],
      "envVars": [
        { "name": "GITHUB_TOKEN", "mode": "deny" },
        { "name": "NPM_TOKEN", "mode": "deny" }
      ]
    }
  }
}
```

Each entry carries `"mode": "deny"`, which is the only supported value. The explicit `mode` field keeps the schema forward-compatible with future modes. File paths follow the same **prefix rules** as `sandbox.filesystem.*` settings, and entries from every **settings scope** are merged. Because the only mode is `deny`, any scope can add restrictions but none can remove them.

There is no built-in credential deny list, so only the files and variables you list are restricted. The setting affects sandboxed Bash commands only. To strip Anthropic and cloud provider credentials from all subprocesses regardless of sandboxing, set `CLAUDE_CODE_SUBPROCESS_ENV_SCRUB`.

How sandboxing works

Filesystem isolation

The sandboxed Bash tool restricts file system access to specific directories:

- **Default write behavior:** read and write access to the current working directory and its subdirectories, plus the session temp directory that `$TMPDIR` points to
- **Default read behavior:** read access to the entire computer, except certain denied directories.

Note that this default still allows reading credential files such as `~/.aws/credentials` and `~/.ssh/`. Use `sandbox.credentials` to block reads of these files and unset secret environment variables, or add the paths to `denyRead`.

- **Blocked access:** cannot modify files outside the current working directory and session temp directory without explicit permission, including shell configuration files such as `~/.bashrc` and system binaries in `/bin/`
- **Git worktrees:** when the working directory is a linked git worktree, the sandbox also allows writes to the main repository's shared `.git` directory so commands such as `git commit` can update refs and the index. Writes to `hooks/` and `config` inside that directory remain denied.
- **Configurable:** define custom allowed and denied paths through settings

You can grant write access to additional paths using `sandbox.filesystem.allowWrite` in your settings. These restrictions are enforced at the OS level, so they apply to all subprocess commands, including tools like `kubectl`, `terraform`, and `npm`, not just Claude's file tools.

Network isolation

Network access is controlled through a proxy server running outside the sandbox:

- **Domain restrictions:** no domains are pre-allowed. The first time a command needs a new domain, Claude Code prompts for approval. As of v2.1.191, choosing Yes allows the host for the rest of the current session, so later connections to the same host do not prompt again. Pre-allow domains with `allowedDomains` to avoid the prompt entirely.
- **Managed lockdown:** if `allowManagedDomainsOnly` is set in managed settings, non-allowed domains are blocked automatically instead of prompting, and only `allowedDomains` from managed settings are honored.
- **Custom proxy support:** advanced users can implement custom rules on outgoing traffic
- **Comprehensive coverage:** restrictions apply to all scripts, programs, and subprocesses spawned by commands

ⓘ The built-in proxy enforces the allowlist based on the requested hostname and does not terminate or inspect TLS traffic. See [Security limitations](#) for the implications of this design, and [Custom proxy configuration](#) if your threat model requires TLS inspection.

OS-level enforcement

The sandboxed Bash tool leverages operating system security primitives:

- **macOS:** uses Seatbelt for sandbox enforcement
- **Linux:** uses **bubblewrap** for isolation
- **WSL2:** uses bubblewrap, same as Linux

WSL1 is not supported because bubblewrap requires kernel features only available in WSL2. These OS-level restrictions ensure that all child processes spawned by Claude Code’s commands inherit the same security boundaries.

These same primitives are available as the standalone `@anthropic-ai/sandbox-runtime` package, which the **Sandbox environments** page covers as a separate approach for wrapping the entire Claude Code process.

How sandboxing relates to permissions and permission modes

Sandboxing, **permission rules**, and **permission modes** are complementary layers. The sections below cover how the sandbox interacts with each.

Permission rules

Permission rules and sandboxing control different things:

- **Permission rules** control which tools Claude Code can use and are evaluated before any tool runs. They apply to all tools: Bash, Read, Edit, WebFetch, MCP, and others.
- **Sandboxing** provides OS-level enforcement that restricts what Bash commands can access at the filesystem and network level. It applies only to Bash commands and their child processes.

The two layers also differ in how they are enforced. Claude Code evaluates permission decisions before a command runs, based on the command string and, in auto mode, a separate classifier’s judgment about whether the command is safe. The operating system enforces the sandbox boundary on the running process, so it holds regardless of what the model chose to run and even if an allowed command does more than its name suggests.

Filesystem and network restrictions are configured through both sandbox settings and permission rules:

Setting or rule	What it does
<code>sandbox.filesystem.allowWrite</code>	Grants subprocess write access to paths outside the working directory

Setting or rule	What it does
<code>sandbox.filesystem.denyWrite</code> and <code>sandbox.filesystem.denyRead</code>	Block subprocess access to specific paths
<code>sandbox.filesystem.allowRead</code>	Re-allows reading specific paths within a <code>denyRead</code> region
Edit <code>allow</code> rules	Grant write access to specific paths, the same way <code>sandbox.filesystem.allowWrite</code> does
Read and Edit <code>deny</code> rules	Block access to specific files or directories
<code>WebFetch</code> allow and deny rules	Control domain access
Sandbox <code>allowedDomains</code>	Controls which domains Bash commands can reach
Sandbox <code>deniedDomains</code>	Blocks specific domains even when a broader <code>allowedDomains</code> wildcard would otherwise permit them

Paths from both `sandbox.filesystem` settings and permission rules are merged together into the final sandbox configuration.

The [claude-code repository's examples directory](#) includes starter settings configurations for common deployment scenarios, including sandbox-specific examples. Use these as starting points and adjust them to fit your needs.

Permission modes

`/sandbox` is not a **permission mode**. Permission modes decide whether a tool call runs and whether you are prompted first, while the sandbox restricts what a Bash command can access once it runs. They differ in what they control and what replaces the per-action prompt:

	What it controls	What replaces the prompt
<code>/sandbox</code>	What a Bash command can access once it runs	The sandbox boundary itself, in <u>auto-allow mode</u>
<u>Auto mode</u>	Whether each tool call runs	A classifier that reviews actions
<code>--dangerously-skip-permissions</code>	Whether each tool call runs	Nothing. <u>Protected path</u> checks are also skipped; only explicit <u>ask rules</u> and removing <code>/</code> or your home directory still prompt

The sandbox's **auto-allow mode** is separate from **auto mode**: auto-allow approves Bash commands because the sandbox boundary contains them, while auto mode uses a classifier to review actions. The two work independently and can be combined. To choose an isolation boundary for unattended

runs, see [Sandbox environments](#).

Configure the sandbox for your organization

Administrators can require sandboxing for every user, keep developers from widening the policy, and route sandbox traffic through a corporate proxy.

Enforce sandboxing with managed settings

To require the sandbox for every developer, deliver the `sandbox` keys through [managed settings](#), either as a file managed by your MDM or through [server-managed settings](#) on Claude.ai.

The following managed settings configuration enables the sandbox, refuses to start Claude Code if the sandbox cannot initialize, and prevents the model from retrying commands outside the sandbox:

```
{
  "sandbox": {
    "enabled": true,
    "failIfUnavailable": true,
    "allowUnsandboxedCommands": false
  }
}
```

The two keys beyond `enabled` control what happens when the sandbox cannot run a command:

- `failIfUnavailable` : a missing dependency such as bubblewrap on Linux blocks Claude Code from starting rather than showing a warning and falling back to unsandboxed execution
- `allowUnsandboxedCommands: false` : the `dangerouslyDisableSandbox` escape hatch is ignored, so commands that fail under the sandbox cannot be retried outside it

Two additions are worth considering alongside them. Add `excludedCommands` for any organization-approved tools that must run without isolation. Add [sandbox.credentials](#) entries for credential directories such as `~/.aws` and `~/.ssh` and for secret environment variables, since the default read policy still allows them.

The sandbox does not run on native Windows, so if your fleet includes Windows hosts, scope this configuration to macOS and Linux or have those users run Claude Code inside WSL2 or a container.

Keep developers from widening the policy

For boolean keys such as `enabled` and `failIfUnavailable`, Claude Code uses the managed value and ignores anything a developer sets locally. For array keys such as `excludedCommands` and `allowRead`, Claude Code merges entries from every scope, so a developer can append entries that widen the policy.

Set `allowManagedReadPathsOnly` to `true` in managed settings so that only `allowRead` entries from managed settings are honored. User, project, and local `allowRead` entries are ignored. This prevents developers from widening read access beyond the organization-approved paths. To lock network domains to the managed values the same way, set `allowManagedDomainsOnly`.

`excludedCommands` has no equivalent managed-only lockdown, so a developer can always append entries that run additional commands outside the sandbox. Keep the managed list narrow.

Custom proxy configuration

For organizations requiring advanced network security, you can implement a custom proxy to:

- Decrypt and inspect HTTPS traffic
- Apply custom filtering rules
- Log all network requests
- Integrate with existing security infrastructure

To point Claude Code at your proxy, set the proxy ports in sandbox settings:

```
{
  "sandbox": {
    "network": {
      "httpProxyPort": 8080,
      "socksProxyPort": 8081
    }
  }
}
```

Troubleshooting

Some commands fail inside the sandbox even though they work outside it. The fixes below cover the most common cases.

- **Commands fail with a host-not-allowed error:** many CLI tools need to reach specific hosts. Granting permission when prompted adds the host to your allowed list so the tool runs inside the sandbox in future.
- `jest` **hangs or fails:** `watchman` is incompatible with the sandbox. Run `jest --no-watchman` instead.
- **Go-based CLIs fail TLS verification on macOS:** tools such as `gh`, `gcloud`, and `terraform` may fail TLS verification under Seatbelt. List these tools in `excludedCommands` to run them outside the sandbox. If you are using `httpProxyPort` with a MITM proxy and custom CA, set `enableWeakerNetworkIsolation` to `true` instead.
- `open`, `osascript`, or **browser-based auth flows fail with error -600 on macOS:** the sandbox blocks Apple Events by default. Set `allowAppleEvents` to `true` in your user, managed, or CLI settings to allow them. Project settings are ignored for this key. Enabling it removes code-execution isolation, since sandboxed commands can then launch other applications unsandboxed with no user prompt and send AppleScript commands to running applications, subject to the macOS automation-consent prompt (TCC). Alternatively, add the command to `excludedCommands` to run it outside the sandbox.
- `docker` **commands fail:** `docker` is incompatible with the sandbox. Add `docker *` to `excludedCommands` to run it outside the sandbox.
- **Bubblewrap fails to start inside a container:** in an unprivileged container, bubblewrap cannot mount a fresh `/proc` filesystem. Set `enableWeakerNestedSandbox` to `true` so the inner sandbox bind-mounts the container's existing `/proc` instead. Only use this setting when the outer container already provides the isolation boundary you need, since it exposes process information to sandboxed commands that a fresh `/proc` mount would hide.
- **Seccomp filter on Linux:** the seccomp filter is required to block Unix domain sockets. The Dependencies tab in `/sandbox` shows whether it is available. If it is missing, run `npm install -g @anthropic-ai/sandbox-runtime` to install the helper.
- `--dangerously-skip-permissions` **fails as root:** this flag is blocked when running as root or via `sudo` on Linux and macOS, because root access combined with no permission prompts can modify any file or service on the system. The check is skipped automatically inside a recognized sandbox. To run autonomously in a container, use the **dev container** configuration, which runs Claude Code as a non-root user.

Limitations

Sandboxing reduces risk but is not a complete isolation boundary. Review the limitations below before relying on it as a hard security control.

Security limitations

- **Network filtering:** the network filtering system operates by restricting the domains that processes are allowed to connect to. The built-in proxy does not terminate or perform TLS inspection on outbound traffic, so the contents of encrypted connections are not examined. You are responsible for ensuring that only trusted domains are allowed in your policy.

⚠️ Allowing broad domains such as `github.com` can create paths for data exfiltration. Because the proxy makes its allow decision from the client-supplied hostname without inspecting TLS, code running inside the sandbox can potentially use domain fronting or similar techniques to reach hosts outside the allowlist. If your threat model requires stronger guarantees, configure a custom proxy that terminates TLS and inspects traffic, and install its CA certificate inside the sandbox. Stronger TLS-aware network isolation is an active area of development.
- **Privilege escalation via Unix sockets:** the `allowUnixSockets` configuration can inadvertently grant access to powerful system services that could lead to sandbox bypasses. For example, allowing access to `/var/run/docker.sock` effectively grants access to the host system through the Docker socket. Consider carefully any Unix sockets that you allow through the sandbox.
- **Filesystem permission escalation:** overly broad filesystem write permissions can enable privilege escalation attacks. Allowing writes to directories containing executables in `$PATH`, system configuration directories, or user shell configuration files such as `.bashrc` or `.zshrc` can lead to code execution in different security contexts when other users or system processes access these files.
- **Linux sandbox strength:** the Linux implementation provides strong filesystem and network isolation but includes an `enableWeakerNestedSandbox` mode that enables it to work inside Docker environments without privileged namespaces, or on Linux hosts where unprivileged user namespaces are disabled by `sysctl`. This option considerably weakens security and should only be used when additional isolation is otherwise enforced.
- **Apple Events on macOS:** the macOS sandbox blocks Apple Events by default. The `allowAppleEvents` setting lifts this restriction so tools such as `open` and `osascript` work, but it removes code-execution isolation: sandboxed commands can launch other applications unsandboxed with no user prompt, and can send AppleScript commands to running applications, subject to the per-app macOS automation-consent prompt (TCC). It is only honored from user, managed, or CLI settings. Project settings cannot enable it.
- **Settings files protected:** the sandbox automatically denies write access to Claude Code's `settings.json` files at every scope and to the managed settings directory, so a sandboxed command cannot modify its own policy.

Platform and tool compatibility

- **Platform support:** supports macOS, Linux, and WSL2. WSL1 and native Windows are not supported.
- **Performance overhead:** minimal, but some filesystem operations may be slightly slower.
- **Tool compatibility:** some tools that require specific system access patterns may need configuration adjustments, or may need to be run outside the sandbox.

Scope

The sandbox isolates Bash subprocesses. Other tools operate under different boundaries:

- **Built-in file tools:** Read, Edit, and Write use the permission system directly rather than running through the sandbox. See [permissions](#).
- **Computer use:** when Claude opens apps and controls your screen, it runs on your actual desktop rather than in an isolated environment. Per-app permission prompts gate each application. See [computer use in the CLI](#) or [computer use in Desktop](#).
- **Environment variables:** sandboxed Bash commands inherit the parent process environment by default, including any credentials set there. Use `sandbox.credentials` to unset specific variables for sandboxed commands, or set `CLAUDE_CODE_SUBPROCESS_ENV_SCRUB` to strip Anthropic and cloud provider credentials from all subprocesses.
- **Subagents:** [subagents](#) run in the same process as the parent session and use the same sandbox configuration. Bash commands inside a subagent are sandboxed when sandboxing is enabled in the parent session.

⚠ Effective sandboxing requires both filesystem and network isolation. Without network isolation, a compromised agent could exfiltrate sensitive files like SSH keys. Without filesystem isolation, a compromised agent could backdoor system resources to gain network access. When you widen the defaults, check that an `allowWrite` path, a broad `allowedDomains` entry, or an `excludedCommands` exception does not undo a restriction on the other side.

See also

- [Sandbox environments](#): compare the built-in sandbox with dev containers, containers, and VMs
- [Security](#): comprehensive security features and best practices
- [Permissions](#): permission configuration and access control
- [Settings](#): complete configuration reference

- CLI reference: command-line options